Hardware-Friendly Robust Threshold ECDSA in an Asymmetric Model

Hankyung Ko[1], Seunghwa Lee[2*], Sookyung Eom[3] and Sung-Hyun Jo[4†]

[1] Crossbar-inc, hankyung.ko@crossbar-inc.com
[2] Crossbar-inc, seung-hwa.lee@crossbar-inc.com
[3] Crossbar-inc, sookyung.eom@crossbar-inc.com
[4] Crossbar-inc, sung-hyun.jo@crossbar-inc.com

Abstract. We propose Asymmetric Robust Threshold ECDSA (**ART-ECDSA**), a robust and hardware-friendly threshold ECDSA protocol designed for asymmetric settings where one participant is a resource-constrained hardware device. The scheme achieves full robustness and cheater identification while minimizing the computational and communication burden on the hardware signer. Our design leverages Castagnos–Laguillaumie (CL) homomorphic encryption to replace Paillier-based operations and remove costly range proofs, yielding compact ciphertexts and simple zero-knowledge proofs. All heavy multiparty computations, including multiplicative-to-additive (MtA) conversions and distributed randomness generation, are offloaded to online cosigners, allowing the hardware party to remain lightweight. ART-ECDSA provides an efficient asymmetric signing protocol with formal security proofs in the UC framework, achieving both robustness and hardware efficiency within a single design.

Our implementation on an ARM Cortex-M7 microcontroller (400 MHz, 3 MB Flash, 2 MB SRAM) shows that the hardware party performs only lightweight computation (50 ms in presigning and ≤ 10 s in signing) and transmits about 300 Bytes and 3 KB in each phase, which easily fits within the bandwidth limits of BLE and NFC. These results demonstrate that ART-ECDSA is practical for cold-storage and embedded hardware environments without compromising security.

Keywords: MPC · hardware-friendly · robust · asymmetry · threshold · ECDSA

1 Introduction

The Elliptic Curve Digital Signature Algorithm (ECDSA) [JMV01] underpins the security of most blockchain systems and digital assets. In Bitcoin, Ethereum, and many other cryptocurrencies, every transaction must be signed with an ECDSA key to be considered valid. Thus, control of the private signing key directly translates into control of the associated assets. As a result, protecting ECDSA signing keys against theft or misuse has become a cornerstone of blockchain security infrastructure.

A natural defense against single points of failure is to distribute trust: instead of storing a key on a single machine, multiple parties each hold a share of the secret key and must cooperate to produce a signature. This approach, known as threshold ECDSA (tECDSA) [GG18, CGG+20, DKLS18, DKLS19, DKLS24, TX25], allows any subset of at least t out of n parties to jointly produce a valid ECDSA signature, while fewer than t parties learn nothing. Threshold signatures thus provide strong protection against key

*Co-corresponding author.
†Co-corresponding author.

compromise, insider threats, and physical theft, and have become central to institutional custody platforms, exchanges, and wallets.

Unlike Schnorr or BLS signatures, which are linear in the secret key, ECDSA's signing equation $s = k^{-1}(m + r \cdot x) \pmod q$ introduces a rational dependency on both the nonce k and private key x. Extending this operation to a multiparty setting requires two cryptographically hard sub-protocols: (i) secure inversion of the shared nonce k, and (ii) secure multiplication of secret shares ($k^{-1} \cdot m$, $k^{-1} \cdot r \cdot x$) without revealing k or x. The core primitive here is the *Multiplication-to-Additive (MtA)* conversion: given multiplicative shares (a, b), parties must obtain additive shares (α, β) such that $\alpha + \beta = ab \bmod q$.

Paillier-based designs [GG18, GG20] realize MtA using additively homomorphic encryption with zero-knowledge range proofs, offering strong security but at high computational and communication cost. These "GG-style" protocols long defined the state-of-the-art for tECDSA, thanks to their modular structure and clear proof framework. However, in 2023, **CVE-2023-33241** revealed a critical vulnerability: a malicious party can inject a crafted Paillier modulus and exploit flaws in range proofs to extract others' secrets within a few signing sessions, undermining even honest-majority settings [MYG24]. Subsequently, the work of [CGG+20] proposed a patched variant that strengthens parameter validation and enforces consistency checks on the Paillier modulus to mitigate this attack. Despite this fix, this incident motivated a reevaluation of robustness and leakage resistance in threshold ECDSA protocols.

Recent works such as [DKLS24] replaced homomorphic encryption with oblivious transfer (OT) and its vector extension (VOLE) to achieve a three-round design. These schemes remove range proofs and achieve statistical security via "consistency checks," reducing rounds but introducing enormous bandwidth overhead-hundreds of kilobytes per presignature when n grows-which is impractical for constrained devices or BLE/NFC-based wallets.

The TX25 protocol [TX25] followed the same three-round paradigm but replaced OT/VOLE with MtA over the Castagnos-Laguillaumie (CL) cryptosystem, drastically reducing bandwidth. Each MtA step carries compact ciphertexts and short proofs, yielding strong efficiency and robustness. TX25 further introduced two key innovations: (i) an *undetermined temporary nonce* γ, ensuring correctness even under aborts, and (ii) *publicly checkable MtA* proofs for cheater identification. These features made TX25 both efficient and robust-but it remained *symmetric*, requiring all parties to perform the same heavy computations, unsuitable for hardware wallets with strict constraints.

Practical motivation: deployment and constraints. Existing consumer MPC wallets (e.g., ZenGo, Fireblocks, Silence Laboratories, Coinbase, Binance) operate in a single-user setting but typically avoid including a cold-storage or resource-constrained hardware device (e.g., Ledger/Trezor) as an MPC party due to limited computation and bandwidth. Consequently, MPC is often performed only between a mobile device and the provider's cloud infrastructure. This architecture introduces two limitations: (i) because one MPC party is operated by the provider, user sovereignty over the signing process is restricted, as the cloud enforces provider-defined policy and approval logic; and (ii) since the mobile device naturally initiates signing, the cloud may approve signatures without an independent verification step under the user's control, weakening the intended multi-party assurance. Our work instead targets a setting where one of the user's devices is an independent, offline, bandwidth-limited cold-storage component that must actively participate in threshold ECDSA signing.

This deployment choice is constrained by both communication and computation. Cold-storage interfaces such as Bluetooth Low Energy (BLE) and Near Field Communication (NFC) impose strict bandwidth limits: while the BLE 5.0 physical layer supports up to 2 Mbps, the effective application throughput is typically 0.27–1.37 Mbps due to protocol overhead, connection intervals, and MTU limits, so even a few hundred kilobytes of inter-

Table 1: Comparison of representative threshold ECDSA protocols along the design-space triangle: robustness, asymmetry, rounds, and bandwidth (5-of-5 setting).

Scheme	Robust	Asym.	# Rounds (Pre/Sign)		Bandwidth (Pre/Sign)	
			HW	Other parties	HW	Other parties
[CGG+20]/Paillier	✗	✗	(3/1)	(3/1)	(93 KB/160 Bytes)	(93 KB/160 Bytes)
[DKLS24]/OT	✗	✗	(2/1)	(2/1)	(1.06 MB/300 Bytes)	(1.06 MB/300 Bytes)
[BMP22]/Asym	✗	✓	(1/1)	(1/4)	(16 KB/2.73 KB)	(12.86 KB/73.56 KB)
[TX25]/CL	✓	✗	(2/1)	(2/1)	(148.00 KB/4.11 KB)	(148.00 KB/4.11 KB)
ART-ECDSA (ours)	✓	✓	**(1/2)**	**(3/3)**	**(300 Bytes/2.98 KB)**	**(62.65 KB/12.28 KB)**

active traffic can cause latency or energy spikes in hardware wallets. Moreover, hardware wallets are often implemented as resource-constrained secure elements or microcontrollers with limited memory and compute (e.g., ARM Cortex-M or RISC-V below 200 Mhz with only tens of kilobytes of RAM and no hardware support for arbitrary-precision arithmetic). Intensive operations such as modular exponentiation, homomorphic encryption, or zero-knowledge proofs can therefore dominate the power budget and signing latency. For these reasons, prior threshold ECDSA schemes, which impose heavy online computation or high communication overhead, are impractical in this setting. ART-ECDSA is explicitly engineered to keep the hardware participant lightweight while preserving robustness and cheater identification.

Design-space triangle. Asymmetric robust threshold ECDSA protocols for resource-constrained hardware devices must simultaneously balance *robustness*, *efficiency*, and *asymmetry*. Robustness ensures that honest parties can always complete a signature despite aborting or malicious participants, and enables cheater identification for fault recovery. Efficiency covers both communication (bandwidth or number of rounds) and local computation, which are critical for BLE/NFC-based hardware wallets. Asymmetry reflects the division of labor between a resource-constrained hardware signer and more powerful online cosigners.

Despite significant progress in threshold ECDSA, prior schemes remain unsatisfactory for hardware-constrained settings. Existing designs fall into three main categories. (i) Paillier-based protocols [GG18, CGG+20] achieve strong security but require heavy range proofs and multiple online rounds, which are hardware-unfriendly. (ii) OT/VOLE-based approaches [DKLS24] reduce the number of rounds to three but suffer from bandwidth explosion, requiring hundreds of KB or even MBs per presignature when n grows. (iii) Asymmetric models such as [BMP22] successfully minimize device workload but lack robustness and cheater identification (CI), while symmetric robust designs like [TX25] assume equal participation by all parties, making them unsuitable for constrained hardware.

Our approach. We design an *asymmetric* tECDSA where *all* MtA and proof-heavy work is shifted to the online cosigners, and the hardware party performs only lightweight steps. To reconcile robustness with hardware asymmetry, we redesign the threshold ECDSA workflow so that all cryptographically heavy sub-protocols—including multiplicative-to-additive (MtA) conversions, distributed randomness generation (DRG), and zero-knowledge proofs—are executed entirely by the online cosigners. The hardware device P_0 participates only through a few constant-size messages and lightweight exponentiations.

The key technical enabler is the α-*handle* mechanism: P_0 samples a random scalar α and publishes $H = G^{\alpha^{-1}}$, which allows the online parties to derive the signing nonce as $R = H^k = G^{k/\alpha}$. This algebraic linkage shifts all nonce- and MtA-related computations to the cosigners, while P_0 later multiplies by α to obtain a valid ECDSA signature. Correctness and cheater identification are guaranteed through compact CL-based NIZKs, which replace the range-proof-heavy Paillier structure with the Castagnos-Laguillaumie (CL) encryption system, yielding constant-size ciphertexts and publicly verifiable proofs.

This design decouples the hardware party from the multi-round, high-bandwidth

interactions in prior robust schemes such as [TX25], yet retains the same formal security and robustness guarantees. As a result, ART-ECDSA achieves a hardware workload of a small constant number of elliptic-curve operations and a few kilobytes communication per signature, while retaining the same formal robustness guarantee within the UC framework.

Our Contributions:

- **Robust asymmetric threshold ECDSA.** We present **ART-ECDSA**, the first robust threshold ECDSA protocol explicitly designed for asymmetric settings, where a single hardware signer remains lightweight while all heavy multiparty computation is offloaded to online cosigners. The protocol guarantees robustness and cheater identification (CI) with only a few kilobyte communication, making it practical for BLE/NFC-constrained environments.
- **Formal security analysis.** We formalize an asymmetric participation model that clearly defines the interfaces between the hardware party P_0 and the online cosigners. Under standard assumptions (DDH, CL-IND-CPA, EUF-CMA), we prove that ART-ECDSA achieves EUF-CMA security, robustness, and CI in the UC framework.
- **Efficient presigning and lightweight signing.** Expensive operations such as MtA conversions and distributed randomness generation are shifted to the presigning phase. The signing path then involves only a single lightweight interaction with P_0, ensuring minimal latency and constant-time operations on hardware.
- **Implementation and benchmarks.** We implement the full protocol. The experimental results confirm that the hardware workload is limited to a few milliseconds and that total communication per presign/sign session fits well within BLE/NFC limits, outperforming prior schemes.

1.1 Paper Organization

The remainder of this paper is organized as follows. Section 2 introduces the necessary preliminaries. Section 3 presents our two-party scheme, covering the detailed protocols followed by their formal security proofs. Section 4 describes our main construction, the **ART-ECDSA** scheme, in the general multi-party setting. It provides full specifications along with the corresponding security analysis and proofs. Section 5 reports our experimental evaluation and benchmark results, demonstrating the efficiency and hardware-friendliness of the proposed protocol. Finally, Section 6 concludes the paper.

2 Preliminaries

2.1 Notation

We write $\mathbb{Z}_q$ for the field of order q. For a set S, $|S|$ denotes its cardinality, and $s \xleftarrow{\$} S$ means s is sampled uniformly from S. For $a \in \mathbb{N}$, $[a] := \{1, \ldots, a\}$. We use $\epsilon(\lambda)$ for a negligible function in the security parameter λ, $\|$ for string concatenation, and $\overset{c}{\approx}$ for computational indistinguishability. Unless stated otherwise, all scalar equalities are modulo q.

There is one hardware party P_0 and cosigners $P_1, \ldots, P_{n-1}$; we write $\infty := [n] \setminus \{0\}$. In a signing session, $S \subseteq \infty$ denotes the set of online participants. For interpolation, we use Lagrange coefficients

$$\lambda_i := \prod_{\substack{j \in S \\ j \neq i}} \frac{-j}{i-j} \in \mathbb{Z}_q,$$

so that for any polynomial f of degree $< |S|$ we have $f(0) = \sum_{i \in S} \lambda_i f(i)$.

2.2 Castagnos-Laguillaumie Encryption

We denote by Π_{CL} the Castagnos–Laguilaumie (CL) encryption scheme [CL15]. It is a linearly homomorphic public-key encryption defined over class groups of unknown order.

Let $\mathbb{G}$ be a class group containing a unique cyclic group $\mathbb{F} = \langle f \rangle$ of prime order q, where q is the order of the elliptic curve group used in ECDSA. The generator $g_q \in \mathbb{G}$ is chosen such that $\mathbb{G} = \mathbb{F} \times \langle g_q \rangle$, and a discrete logarithm of f with respect to g_q is unknown. Message $m \in \mathbb{Z}_q$ are encoded as $f^m \in \mathbb{F}$.

Π_{CL} consists of the following algorithms:

- $\mathsf{KeyGen}(1^\lambda) \to (sk, pk)$: sample a secret key sk from distribution $\mathcal{D}_q$, and compute the public key $pk = g_q^{sk}$.
- $\mathsf{Enc}(pk, m) \to c$: on input a message $m \in \mathbb{Z}_q$, choose randomness $r \xleftarrow{\$} \mathcal{D}_q$, and output ciphertext $c = (g_q^r, f^m pk^r)$.
- $\mathsf{Dec}(sk, c) \to m$: parse $c = (c_1, c_2)$, and recover $m = \mathsf{SolveDLog}_f(c_2/c_1^{sk})$.

The scheme is additively homomorphic: for ciphertexts c_1, c_2 and $k \in \mathbb{Z}_q$,

$$\Pi_{\mathrm{CL}}.\mathrm{Enc}(m_1) \cdot \Pi_{\mathrm{CL}}.\mathrm{Enc}(m_2) = \Pi_{\mathrm{CL}}.\mathrm{Enc}(m_1 + m_2), \qquad \Pi_{\mathrm{CL}}.\mathrm{Enc}(m)^k = \Pi_{\mathrm{CL}}.\mathrm{Enc}(k \cdot m).$$

The semantic security is based on the *Decisional Diffie–Hellman* (DDH) assumption over ideal class groups.

2.3 Zero-Knowledge Proof

We use Fiat-Shamir non-interactive zero-knowledge (FS-NIZK) proofs in the random-oracle model (ROM), obtained from Schnorr-style Σ-protocols for group homomorphisms.

Required Properties. For each relation $\mathcal{R}$, we require:

- **Completeness:** An honest prover with a valid witness always produces an accepting proof.
- **Soundness:** No PPT prover can produce an accepting proof for a false statement except with negligible probability.
- **Zero-Knowledge:** There exists a PPT simulator that, given only the statement, produces proofs indistinguishable from real proofs.
- **Simulation-Extractability (for $R_{\mathbf{key}}$, $R_{\mathbf{Enc\text{-}DL}}$):** For relations where the simulator must extract witness from adversarial proofs, we require simulation-extractability in the ROM.

Relations used in our scheme. We instantiate FS-NIZKs for the following relations, whose roles in our protocol are summarized in Table 2.

$$\mathcal{R}_{\mathrm{key}} = \{\, (\mathrm{ek}; \mathrm{dk}) \mid \mathrm{dk} \in \mathcal{D},\ \mathrm{ek} = \mathfrak{g}_q^{\mathrm{dk}} \,\}.$$

$$\mathcal{R}_{\mathrm{sch}} = \{\, (X; x) \mid X = G^x \,\}.$$

$$\mathcal{R}_{\mathrm{Sh}} = \{\, (\{C_{f(j)}\}_{j\in S}, \{\mathrm{ek}_j\}_{j\in S};\ f(\cdot), \{\rho_j\}_{j\in S}) \mid \deg(f) \le t-1,\ C_{f(j)} = \Pi_{\mathrm{CL}}.\mathrm{Enc}_{\mathrm{ek}_j}(f(j); \rho_j) \,\}.$$

$$\mathcal{R}_{\text{Dec-DL}} = \{\, ((c_1, c_2), A, \mathrm{ek};\ a, \mathrm{dk}) \mid A = G^a,\ a = \Pi_{\mathrm{CL}}.\mathrm{Dec}_{\mathrm{dk}}(c_1, c_2) \,\}.$$

$$\mathcal{R}_{\text{Enc-DL}} = \{\, (C, A, \mathrm{ek};\ a, \rho) \mid A = G^a,\ C = \Pi_{\mathrm{CL}}.\mathrm{Enc}_{\mathrm{ek}}(a; \rho) \,\}.$$

$$\mathcal{R}_{\chi} = \left\{ \begin{array}{l} C,\ \mathrm{com}_{\chi_1,v},\ \mathrm{com}_{\chi_1},\ \mathrm{com}_v; \\ \chi_1,\ v,\ r,\ \gamma,\ \rho,\ \rho_1,\ \rho_2,\ \rho_{12} \end{array} \middle| \begin{array}{l} v = r\gamma, \\ C = \Pi_{CL} \cdot \Pi_{\mathrm{CL}}.\mathrm{Enc}(\mathrm{ek}; \chi_1, \rho)\ \oplus\ W_0^v, \\ \mathrm{com}_{\chi_1,v} = \Pi_{\text{Ped-vec}} \cdot \mathrm{Commit}(\hat{ck}; (\chi_1, v); \rho_{12}), \\ \mathrm{com}_{\chi_1} = \Pi_{\mathrm{Ped}}.\mathrm{Commit}(ck; \chi_1; \rho_1), \\ \mathrm{com}_v = \Pi_{\mathrm{Ped}}.\mathrm{Commit}(ck; v; \rho_2) \end{array} \right\}.$$

$$\mathcal{R}_{\delta} = \left\{ \begin{array}{l} \mathrm{com}_k,\ \mathrm{com}_\gamma,\ \delta,\ A,\ R; \\ k,\ \gamma \end{array} \middle| \begin{array}{l} \mathrm{com}_k = \Pi_{\mathrm{Ped}} \cdot \Pi_{\mathrm{Ped}}.\mathrm{Commit}(ck; k), \\ \mathrm{com}_\gamma = \Pi_{\mathrm{Ped}} \cdot \Pi_{\mathrm{Ped}}.\mathrm{Commit}(ck; \gamma), \\ \delta = k\gamma, \quad R = A^k \end{array} \right\}.$$

Concrete ZKP constructions for $\mathcal{R}_\chi$ and $\mathcal{R}_\delta$

We provide Schnorr-style Σ-protocols for the relations used in signing.

Proof for $\mathcal{R}_\chi$.

1. **Commit:** Sample $\tilde{\chi}_1, \tilde{v}, \tilde{\gamma} \leftarrow \mathbb{Z}_q$, $\tilde{\rho} \leftarrow \mathcal{D}_q$, $\tilde{\rho}_1, \tilde{\rho}_2, \tilde{\rho}_{12} \leftarrow \mathbb{Z}_q$. Compute and send: $A_1 = g_q^{\tilde{\rho}}$, $A_2 = f^{\tilde{\chi}_1} \cdot ek_0^{\tilde{\rho}} \cdot w_2^{\tilde{v}}$, $A_3 = \mathsf{Com}(\tilde{\chi}_1, \tilde{v})$, $A_4 = \mathsf{Com}(\tilde{\chi}_1)$, $A_5 = \mathsf{Com}(\tilde{v})$, $A_6 = \mathsf{Com}(\tilde{\gamma}) \cdot G^{\tilde{v}}$
2. **Challenge:** $e \leftarrow H(\mathsf{stmt} \| A_1, \ldots, A_6)$
3. **Response:** $z_{\chi_1} = \tilde{\chi}_1 + e \cdot \chi_1$, $z_v = \tilde{v} + e \cdot v$, $z_\gamma = \tilde{\gamma} + e \cdot \gamma$, $z_\rho = \tilde{\rho} + e \cdot \rho$, $z_{\rho_1} = \tilde{\rho}_1 + e \cdot \rho_1$, $z_{\rho_2} = \tilde{\rho}_2 + e \cdot \rho_2$, $z_{\rho_{12}} = \tilde{\rho}_{12} + e \cdot \rho_{12}$
4. **Verify:** Check $g_q^{z_\rho} \stackrel{?}{=} A_1 \cdot c_1^e$, $f^{z_{\chi_1}} \cdot ek_0^{z_\rho} \cdot w_2^{z_v} \stackrel{?}{=} A_2 \cdot c_2^e$, and analogous equations for $A_3, \ldots, A_6$.

Proof for $\mathcal{R}_\delta$.

1. **Commit:** Sample $\tilde{k}, \tilde{\gamma}, \tilde{\rho}_k, \tilde{\rho}_\gamma \leftarrow \mathbb{Z}_q$. Send: $A_1 = G^{\tilde{k}} \cdot \tilde{G}^{\tilde{\rho}_k}$, $A_2 = G^{\tilde{\gamma}} \cdot \tilde{G}^{\tilde{\rho}_\gamma}$, $A_3 = H^{\tilde{k}}$, $A_4 = \tilde{k} \cdot \gamma + k \cdot \tilde{\gamma} \mod q$
2. **Challenge:** $e \leftarrow H(\mathsf{stmt} \| A_1, \ldots, A_4)$
3. **Response:** $z_k = \tilde{k} + e \cdot k$, $z_\gamma = \tilde{\gamma} + e \cdot \gamma$, $z_{\rho_k} = \tilde{\rho}_k + e \cdot \rho_k$, $z_{\rho_\gamma} = \tilde{\rho}_\gamma + e \cdot \rho_\gamma$
4. **Verify:** Check $G^{z_k} \cdot \tilde{G}^{z_{\rho_k}} \stackrel{?}{=} A_1 \cdot \mathsf{com}_k^e$, $G^{z_\gamma} \cdot \tilde{G}^{z_{\rho_\gamma}} \stackrel{?}{=} A_2 \cdot \mathsf{com}_\gamma^e$, $H^{z_k} \stackrel{?}{=} A_3 \cdot R^e$, $z_k \cdot z_\gamma \stackrel{?}{=} A_4 + e \cdot \delta + e^2 \cdot \delta$

Table 2: Zero-knowledge relations and their roles in ART-ECDSA.

Relation	Phase	Purpose
R_{key}	DKG	Prove correct CL key generation
R_{sch}	DKG, Presign	Prove knowledge of discrete logarithm
R_{Sh}	DKG, Presign	Prove validity of threshold-shared ciphertexts
$R_{\text{Dec-DL}}$	DKG, Presign	Link CL decryption to EC public share
$R_{\text{Enc-DL}}$	DKG	Bind encrypted share to public commitment
R_χ	Sign	Ensure consistency of signature numerator
R_δ	Sign	Ensure consistency of signature denominator

2.4 Distributed Randomness Generation (DRG)

Our scheme employs the distributed randomness generation (DRG) protocol from Tang and Xue [TX25], which is based on the publicly verifiable secret sharing (PVSS) construction over class groups by Cascudo and David [CD24]. The DRG enables each participant P_i to obtain a t-threshold share x_i of a random scalar $x \xleftarrow{\$} \mathbb{Z}_q$, along with the public group element $X = xG$ and verifiable public shares $X_i = x_i G$. The protocol consists of two communication rounds:

1. **Share Distribution:** Each P_i selects a random polynomial $f_i(\cdot)$ of degree $t-1$, encrypts its evaluations $\{f_i(j)\}_{j \in S}$ under the CL public keys $\{\text{ek}_j\}$, and broadcasts ciphertexts $\{C_{f_i(j)}\}_{j \in S}$ along with a zero-knowledge proof π_i that these ciphertexts correspond to a valid polynomial of degree $t-1$.
2. **Share Combination:** Upon verifying all π_i, each P_j decrypts the received ciphertexts $\{C_{f_i(j)}\}_i$ using its secret key dk_j, combines the plaintexts to compute its final share $x_j = \sum_i f_i(j)$, and broadcasts the public value $X_j = x_j G$ with a proof π'_j of correct decryption.

Public verifiability of the CL-based PVSS ensures that any malformed ciphertext or proof can be detected and the corresponding participant is publicly identified as a cheater. This DRG serves as the key generation primitive in our threshold scheme, providing t-threshold shares of the signing key and other random nonces. We refer to [TX25] section 2.4 for the complete protocol specification and security analysis.

2.5 Multiplicative-to-Additive (MtA) Protocol

We also adopt the multi-party Multiplicative-to-Additive (MtA) protocol from [TX25], instantiated with CL encryption to avoid range proofs. Given inputs $a, b \in \mathbb{Z}_q$ held by parties P_A and P_B, the MtA computes additive shares α, β such that $\alpha + \beta = ab \bmod q$. The protocol supports public checking, enabling any observer to verify correctness and thus to identify cheaters. At a high level:

1. P_A encrypts its input a as $C_a = \mathsf{Enc}(\mathrm{ek}_A, a)$ and sends it with a proof of well-formedness.
2. P_B samples a random $\beta \xleftarrow{\$} \mathbb{Z}_q$, computes $C_\alpha = b \otimes C_a \oplus \mathsf{Enc}(\mathrm{ek}_A, -\beta)$, and provides a public ZKP attesting that b corresponds to its public key share $X_B = bG$.
3. P_A decrypts C_α to obtain $\alpha = ab - \beta \bmod q$.

For the multi-party case, each participant P_i with inputs (k_i, γ_i) runs parallel MtAs with all others' ciphertexts $\{C_{\gamma_j}\}$, generating local shares $\{\delta_{i,j}\}$ such that $\sum_{i,j} \delta_{i,j} = k\gamma$. [TX25] employ a batching technique to aggregate multiple CL-based ZKPs into a single constant-size proof, allowing efficient verification of all MtA instances simultaneously. The full protocol details is described in [TX25] section 3.

2.6 SecurityModel

Communication Model. We assume a synchronous network where all parties are connected via pairwise authenticated channels that guarantee message integrity and sender authenticity. Additionally, we assume a reliable broadcast primitive that ensures all parties receive identical messages when a party broadcasts. Consequently, any derivation from the protocol execution must originate from a corrupted party. When no party is corrupted, the adversary can only observe the protocol transcript and is therefore passive by definition. This communication model is consistent with prior threshold ECDSA works [GG18, GG20, CGG+20, TX25, DKLS24].

Adversarial Model. We consider a *static* adversary $\mathcal{A}$ who corrupts a fixed set of parties at the beginning of the protocol execution. In the two-party setting, $\mathcal{A}$ may corrupt either P_0 or P_1 (but not both). In the (n, t)-threshold setting, $\mathcal{A}$ may corrupt up to $t - 1$ parties which may include the hardware party P_0. Corrupted parties are under full control of $\mathcal{A}$: the adversary learns their internal states and can send arbitrary messages on their Honest parties follow the protocol specification.

Security Goals. Our protocol aims to UC-realize the ideal threshold ECDSA functionality $\mathcal{F}_{\mathrm{2ECDSA}}$ (Figure 2) against static corruptions. This guarantees the folowiing properties:

- **Unforgeability:** No PPT adversary corrupting fewer than t parties can produce a valid signature on a message m unless all honest parties have agreed to sign m.
- **Privacy:** The protocol transcript reveals nothing about the secret key x individual key shares $\{x_i\}$, or nonces beyond what is implied by the public key X and issued signatures.

3 Two Party Scheme

We first present the core two-party variant of ART-ECDSA between an offline hardware party P_0 and a single online cosigner P_1 (Figure 1). The design follows the asymmetric paradigm of [BMP22]: the online party performs the computation- and proof-heavy tasks, while the hardware party acts as a lightweight decryption/signing module. Our main technical departure from [BMP22] is to replace Paillier–based AHE with Castagnos–Laguillaumie (CL) encryption [CL15], which allows us to avoid range-proof machinery and

significantly reduce both computation and communication costs.

DKG and setup. In the setup phase, P_0 generates a CL keypair (ek_0, dk_0) and samples its secret share x_0, publishing (X_0, W_0) where $X_0 = G^{x_0}$ and $W_0 = \Pi_{\mathrm{CL}}.\mathrm{Enc}(ek_0; x_0)$, together with a ZK proof of correct encryption. In parallel, P_1 samples x_1 and publishes $X_1 = G^{x_1}$ with a Schnorr–style proof of knowledge. To support later consistency checks, P_1 also provides Pedersen commitment parameters $(ck, \hat{ck})$ shared by both parties. Both parties set the joint public key $X = X_0 \cdot X_1$.

Presigning (message-independent). Presigning is message-independent and can be executed and cached ahead of time. The hardware party samples $\alpha \leftarrow \mathbb{F}_q$ and publishes the α-handle $H = G^{\alpha^{-1}}$ with a proof of knowledge of α^{-1}. The online party samples $(k, \gamma) \leftarrow \mathbb{F}_q$ and commits to k, γ, and $\zeta = x_1\gamma$ via Pedersen commitments $\mathrm{com}_k, \mathrm{com}_\gamma, \mathrm{com}_\zeta$. It then derives $R = H^k$ and $r = \mathrm{xcoord}(R) \bmod q$, and sends R to P_0. Intuitively, H lets P_1 form the nonce point while keeping P_0's online work minimal; later, P_0 will reinsert the missing α factor into the final signature. More precisely, when P_1 computes $R = H^k = G^{k/\alpha}$, it obtains a valid nonce point without knowing α. The key observation is that from P_1's perspective, R appears as a random group element since α is uniformly sampled and kept secret by P_0. During singing, P_0 multiplies the decrypted value by α, which effectively "corrects" the nonce from k/α back to k in the signature equation. This yields $s = \alpha \cdot \delta^{-1} \cdot s' = (m + rx)/(k/\alpha \cdot \alpha) = (m + rx)/k$, a valid ECDSA signature with effective nonce $k' = k/\alpha$. The construction ensures that P_0 contributes essential entropy to the signature without performing any MtA or heavy computation.

Signing (single online-to-hardware request). Given a message hash m, P_1 computes $\chi = \gamma(m + rx_1)$ and $v = r\gamma$, commits to (χ, v), and forms the CL ciphertext $C_\chi = \Pi_{\mathrm{CL}}.\mathrm{Enc}(ek_0; \chi) \oplus W_0^v$, which encrypts $\gamma(m + r(x_0 + x_1)) = \gamma(m + rx)$ under P_0's key. It also sets $\delta = k\gamma$. To enable robustness/cheater identification, P_1 proves in zero knowledge that (i) C_χ is well formed and consistent with the Pedersen commitments (relation $\mathcal{R}_\chi$), and (ii) δ is consistent with the committed (k, γ) and the nonce relation $R = H^k$ (relation $\mathcal{R}_\delta$). These two proofs serve complementary roles in ensuring signature validity. The proof π_χ for relation $\mathcal{R}_\chi$ guarantees that the ciphertext C_χ correctly encodes $\gamma(m + rx)$, binding P_1's contribution to the committed values. The proof π_δ for relation $\mathcal{R}_\delta$ establishes that $\delta = k\gamma$ and that $R = H^k$, linking the denominator to the nonce. Together, they ensure that γ appears identically in both numerator and denominator: when P_0 computes $s = s' \cdot \alpha \cdot \delta^{-1} = \alpha \cdot \gamma(m + rx)/(k\gamma)$, the γ terms cancel exactly, yielding $(m + rx)/(k/\alpha)$. Without both proofs, a malicious P_1 could use different γ values in the numerator and denominator, breaking signature validity or leaking information. The hardware party verifies the proofs, decrypts $s' = \Pi_{\mathrm{CL}}.\mathrm{Dec}(dk_0; C_\chi)$, and outputs $s = s' \cdot \alpha \cdot \delta^{-1} \bmod q$. By construction, this yields a valid ECDSA signature with effective nonce $k' = k/\alpha$, while keeping P_0's computation essentially to proof verification, one CL decryption, and a few field operations.

Security intuition (unforgeability and robustness). Unforgeability follows from two complementary mechanisms. First, as long as P_0 is honest, signatures can only be produced via $\Pi_{\mathrm{CL}}.\mathrm{Dec}(dk_0; \cdot)$, and the ciphertext C_χ must pass publicly verifiable ZK proofs tying it to the committed values and to the nonce relation; hence a malicious P_1 cannot induce P_0 to sign with inconsistent (k, γ) or malformed C_χ without being caught. Second, if P_0 is corrupted, the scheme reduces to standard ECDSA signing with an adversarial signer, and security is captured by the ideal functionality's corruption semantics.

Robustness (and cheater identification) is enforced by making all message-dependent contributions of P_1 *publicly checkable*: the proofs π_χ and π_δ certify correctness and consistency of the presignature materials and the signing transcript. Therefore, any deviation that could bias the nonce, reuse k, or cause an invalid signature manifests as a

failed verification, pinpointing the misbehaving party and allowing the protocol to abort safely without compromising the key.

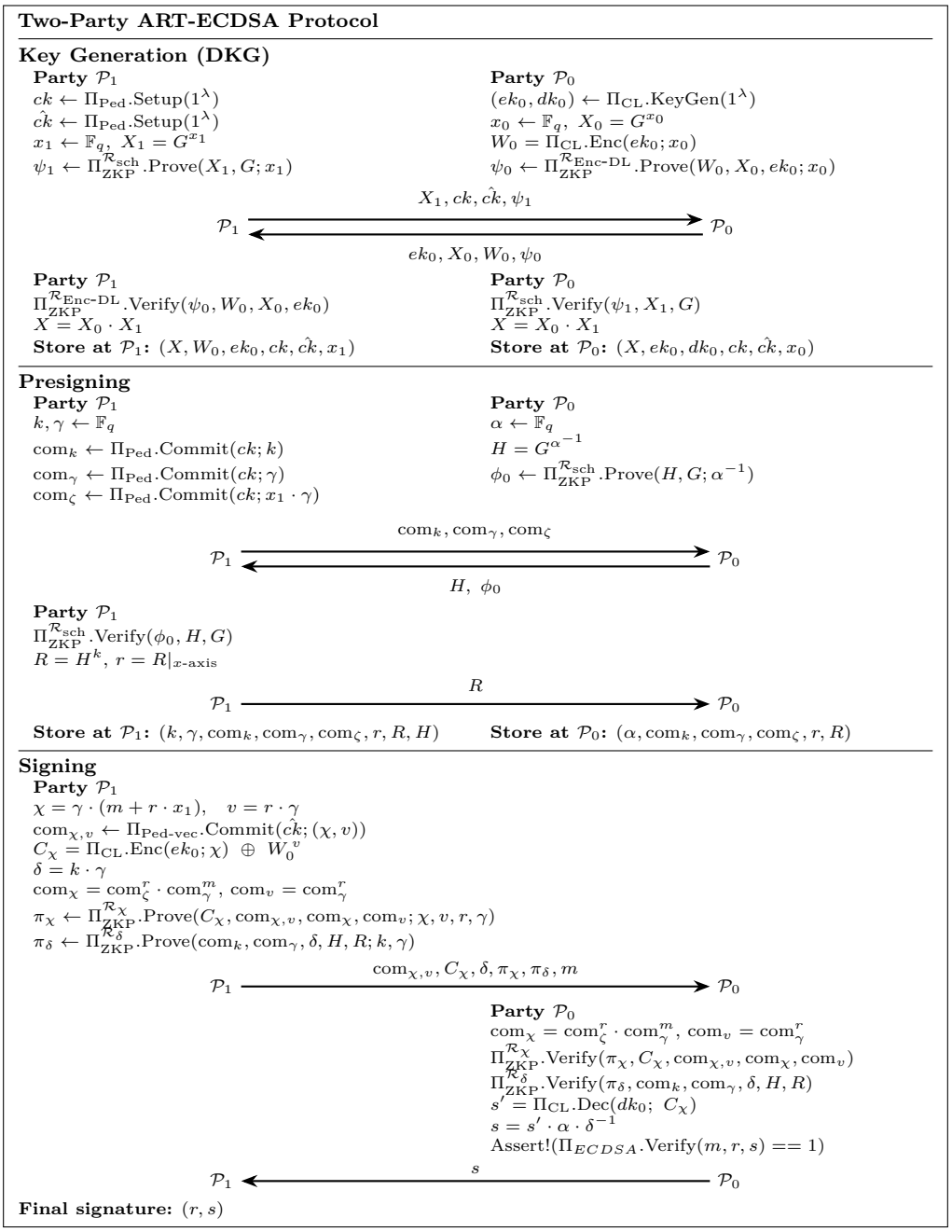

Figure 1: Two-party ART-ECDSA protocol.

3.1 Security Proof

We prove the two party protocol UC-realizes an ideal two-party ECDSA functionality in the Random Oracle Model (ROM) against static corruptions. All zero-knowledge (ZK) subprotocols are Schnorr-type.

3.1.1 Ideal Functionality

The complete interface and behavior of ideal two-party ECDSA functionality $\mathcal{F}_{\mathsf{2ECDSA}}$ with abort are given in Figure 2. Figure 2 is the two-party specialization of the threshold functionality formalism adopted in prior work. We use it verbatim as the target ideal world.

Parties and quorums. There are exactly two parties P_0, P_1. Let $\mathsf{Corrupt} \subset \{P_0, P_1\}$ be the set of corrupted parties (as declared by the adversary $\mathcal{S}$ via corrupt messages at any time; in our setting it is static).

State. Session identifier sid; secret key $x \in \mathbb{Z}_q$; public key $X = G^x$; a map $\mathsf{Req} : \{P_0, P_1\} \to \{\bot\} \cup \mathcal{M}$ recording each party's pending sign request; a set $\mathsf{SigLog} \subseteq \mathcal{M} \times \Sigma$ of issued signatures; a map $\mathsf{PreSig} : \mathsf{PSID} \to \{\mathsf{pending}, \mathsf{ready}, \mathsf{used}\}$ tracking presignature status; a counter $\mathsf{psctr} \leftarrow 0$

Key-generation.

1. Upon receiving $(\mathsf{init}, \mathsf{sid})$ from P_i, record $(\mathsf{init}, \mathsf{sid}, P_i)$.
2. Once $(\mathsf{init}, \mathsf{sid}, P_0)$ and $(\mathsf{init}, \mathsf{sid}, P_1)$ are both recorded: sample $x \leftarrow \mathbb{Z}_q$, set $X \leftarrow G^x$, and output $(\mathsf{pubkey}, \mathsf{sid}, X)$ to P_0, P_1 (and leak it to $\mathcal{S}$).

Presigning.

1. Upon receiving $(\mathsf{presign}, \mathsf{sid}, \mathsf{psid})$ from P_i, record $(\mathsf{presign}, \mathsf{sid}, \mathsf{psid}, P_i)$ and notify $\mathcal{S}$.
2. Once both $(\mathsf{presign}, \mathsf{sid}, \mathsf{psid}, P_0)$ and $(\mathsf{presign}, \mathsf{sid}, \mathsf{psid}, P_1)$ are recorded:
   (a) If $\mathcal{S}$ sends $(\mathsf{set\text{-}nonce}, \mathsf{sid}, \mathsf{psid}, R')$ before timeout, set $R \leftarrow R'$.
   (b) Otherwise, sample $k \leftarrow \mathbb{Z}_q^*$ and set $R \leftarrow G^k$.
   (c) Compute $r \leftarrow R|_{\text{x-axis}} \bmod q$.
   (d) Store (psid, R, r), set $\mathsf{PreSig}(\mathsf{psid}) \leftarrow \mathsf{ready}$.
   (e) Output $(\mathsf{presign\text{-}complete}, \mathsf{sid}, \mathsf{psid}, R)$ to P_0, P_1.
3. If either party aborts, set $\mathsf{PreSig}(\mathsf{psid}) \leftarrow \bot$ and output $(\mathsf{presig\text{-}abort}, \mathsf{sid}, \mathsf{psid})$.

Signing.

1. Upon receiving $(\mathsf{sign}, \mathsf{sid}, \mathsf{psid}, m)$ from P_i, if $\mathsf{PreSig}(\mathsf{psid}) = \mathsf{ready}$, set $\mathsf{Req}(P_i) \leftarrow (\mathsf{psid}, m)$ and notify $\mathcal{S}$.
2. If $\mathsf{Req}(P_0) = (\mathsf{psid}, m)$ and $\mathsf{Req}(P_1) = (\mathsf{psid}, m)$:
   (a) Retrieve (R, r) from stored (psid, R, r).
   (b) Compute $s \leftarrow k^{-1}(m + rx) \cdot \mathsf{DLOG}_R^{-1} \bmod q$ where DLOG_R satisfies $G^{\mathsf{DLOG}_R} = R$.
   (If R was set by $\mathcal{S}$, compute s such that (r, s) verifies under X; if no such s exists, set $s \leftarrow \bot$.)
   (c) Record $(m, (r, s))$ into SigLog, set $\mathsf{PreSig}(\mathsf{psid}) \leftarrow \mathsf{used}$.
   (d) Output $(\mathsf{signature}, \mathsf{sid}, m, (r, s))$ to both parties.
3. If either party never sends $(\mathsf{sign}, \mathsf{sid}, m)$ (or withdraws out-of-band), no signature is produced (abort).

Public verification.

Upon receiving $(\mathsf{sig\text{-}vrfy}, \mathsf{sid}, m, (r, s), X')$ from any external caller $\mathcal{X}$:

1. If $X' \neq X$, set $\beta \leftarrow 0$.
2. Else if $(m, (r, s)) \in \mathsf{SigLog}$, set $\beta \leftarrow 1$.
3. Else if $\mathsf{Corrupt} \neq \{P_0, P_1\}$ (i.e., at least one honest party exists), set $\beta \leftarrow 0$. *[Unforgeability]*
4. Else (both parties corrupted), set $\beta \leftarrow \Pi_{\text{ECDSA}}.\text{Verify}(m, (r, s), X)$ (the real ECDSA verifier).

Output $(\mathsf{istrue}, \mathsf{sid}, m, (r, s), \beta)$ to $\mathcal{X}$ and record $(m, (r, s), \beta)$.

Corruption.

1. Upon receiving $(\mathsf{corrupt}, P_i)$ from $\mathcal{S}$, add P_i to $\mathsf{Corrupt}$.

Remarks. (i) $\mathcal{F}_{\text{2ECDSA}}$ never releases x or any nonce k; only (X) and issued signatures are visible. (ii) Unforgeability is enforced by returning 0 on verification of any pair not in SigLog as long as at least one party is honest. (iii) Abort is modeled by one party withholding its sign request. (iv) Each presignature psid can be consumed at most once; reuse attempts are silently ignored. (v) Presigning and signing are temporally decoupled: a ready presignature may be consumed for any message m

Figure 2: Ideal Two-Party ECDSA Functionality $\mathcal{F}_{\text{2ECDSA}}$

3.1.2 Main Theorem

Theorem 1. *Under DDH, CL-IND-CPA, EUF-ECDSA, and SE-ZKP in the ROM, the protocol realizes $\mathcal{F}_{2ECDSA}$ with abort against static corruptions. Consequently the protocol enjoys* Privacy *and* Unforgeability*.*

We prove indistinguishability between the real execution and the ideal execution with a simulator, considering three cases: (A) malicious P_0, (B) malicious P_1, and (C) passive eavesdropper.

We fix a security parameter λ and let $\epsilon(\cdot)$ denote a negligible function. For a PPT environment $\mathcal{Z}$ interacting with an adversary $\mathcal{A}$ we write $\mathrm{EXEC}^{\mathrm{real}}_{\Pi,\mathcal{A},\mathcal{Z}}(\lambda)$ for the bit output of $\mathcal{Z}$ in the real execution of our protocol Π, and $\mathrm{EXEC}^{\mathrm{ideal}}_{\mathcal{F}_{\mathrm{2ECDSA}},\mathcal{S},\mathcal{Z}}(\lambda)$ for the bit output of $\mathcal{Z}$ in the ideal execution with functionality $\mathcal{F}_{\mathrm{2ECDSA}}$ and simulator $\mathcal{S}$. We must show that for every PPT $\mathcal{A}$ there exists PPT $\mathcal{S}$ such that

$$\Big|\Pr\big[\mathrm{EXEC}^{\mathrm{real}}_{\Pi,\mathcal{A},\mathcal{Z}}(\lambda)=1\big]-\Pr\big[\mathrm{EXEC}^{\mathrm{ideal}}_{\mathcal{F}_{\mathrm{2ECDSA}},\mathcal{S},\mathcal{Z}}(\lambda)=1\big]\Big|\le\epsilon(\lambda).$$

Because corruptions are static, either none is corrupted, or P_0 is corrupted, or P_1 is corrupted. We handle the three cases separately and then combine the bounds by a union bound over the fixed case.

Lemma 1 (Consistency)**.** *If all ZK proofs in the Signing phase verify and $R = H^k$ with $H = g^{\alpha^{-1}}$, then letting $k' := k\alpha^{-1}$ we have*

$$\Pi_{CL}.Dec(\mathsf{dk}_0; C^{\delta^{-1}}) = k^{-1}(m+rx) \quad \textit{and} \quad s = \alpha\cdot k^{-1}(m+rx) = (k')^{-1}(m+rx).$$

Hence (r,s) is a valid ECDSA signature under public key X.

Proof. By correctness of CL and soundness of $\mathcal{R}_\chi$ and $\mathcal{R}_\delta$,

$$\Pi_{\mathrm{CL}}.\mathrm{Dec}(\mathsf{dk}_0; C^{\delta^{-1}}) = \delta^{-1}(\chi + vx_0) = (k\gamma)^{-1}\big(\gamma(m+rx_1)+r\gamma x_0\big) = k^{-1}(m+rx).$$

Multiplying by α gives $s = \alpha k^{-1}(m+rx)$. Since $R = H^k = g^{k\alpha^{-1}} = g^{k'}$, the ECDSA verification equation uses the ephemeral k' and accepts (r,s). □

Case A: Malicious $\mathbf{P_0}$.

Lemma 2. *For every PPT adversary controlling P_0 there exists a PPT simulator $\mathcal{S}_0$ such that for all PPT environments $\mathcal{Z}$, $\big|\Pr[\mathrm{EXEC}^{\mathrm{real}}_{\Pi,\mathcal{A},\mathcal{Z}}=1]-\Pr[\mathrm{EXEC}^{\mathrm{ideal}}_{\mathcal{F}_{2ECDSA},\mathcal{S}_0,\mathcal{Z}}=1]\big|\le\epsilon$.*

Proof. **Simulator $\mathcal{S}_0$.** $\mathcal{S}_0$ internally runs $\mathcal{A}$ (controlling P_0) and simulates the honest P_1. Using simulation-extractability (SE-ZKP), from ψ_0 and ϕ_0 it extracts x_0 and α while producing proofs that are indistinguishable from real ones. Then $\mathcal{S}_0$ samples $x_1 \leftarrow \mathbb{Z}_q$, sets $X_1 = G^{x_1}$ and registers $X = X_0X_1$ with $\mathcal{F}_{\mathrm{2ECDSA}}$ (the environment receives the same X as in the real world).

In the Presigning/Signing phases, $\mathcal{S}_0$ samples $k,\gamma \leftarrow \mathbb{Z}_q$, computes all commitments and ciphertexts honestly as in Π and sends them to $\mathcal{A}$, together with honestly generated ZK proofs. Whenever the environment requests signing of a message m, $\mathcal{S}_0$ follows the protocol with $\mathcal{A}$, obtains the value s output by $\mathcal{A}$ and, at that point, asks $\mathcal{F}_{\mathrm{2ECDSA}}$ to deliver (r,s) to the honest party (i.e., triggers both sign requests on behalf of P_0 and P_1). This step is consistent with Figure 2 because $\mathcal{F}_{\mathrm{2ECDSA}}$ outputs a fixed pair (r,s) to both parties; by Lemma 1, the s computed by $\mathcal{A}$ equals the value that an honest execution would output.

Indistinguishability. (i) By SE-ZKP, the simulated ψ_0, ϕ_0 are indistinguishable from real proofs, while yielding witnesses x_0, α to the simulator. (ii) All values sent by $\mathcal{S}_0$ to $\mathcal{A}$

are distributed exactly as in the real protocol (Pedersen commitments are perfectly hiding; ciphertexts are honestly distributed). (iii) If $\mathcal{A}$ makes the honest party accept, Lemma 1 implies that the resulting (r, s) equals the ECDSA signature under key X, which is precisely what $\mathcal{F}_{\text{2ECDSA}}$ outputs on m. Hence the transcript and outputs in the ideal world are *identically distributed* to those in the real world, except with negligible probability due to ZK soundness failure. Therefore any distinguisher $\mathcal{Z}$ has at most negligible advantage. □

Case B: Malicious $\mathbf{P_1}$.

Lemma 3. *For every PPT adversary controlling P_1 there exists a PPT simulator $\mathcal{S}_1$ such that for all PPT environments $\mathcal{Z}$,*

$$\left|\Pr[\mathsf{EXEC}^{\mathsf{real}}_{\Pi,\mathcal{A},\mathcal{Z}} = 1] - \Pr[\mathsf{EXEC}^{\mathsf{ideal}}_{\mathcal{F}_{2ECDSA},\mathcal{S}_1,\mathcal{Z}} = 1]\right| \leq \epsilon.$$

Proof. **Simulator $\mathcal{S}_1$.** $\mathcal{S}_1$ simulates the honest P_0 to $\mathcal{A}$. It generate fresh CL key-pair (ek_0, dk_0) and set W_0 arbitrarily. Using the ZK simulator, it produces ψ_0 without a witness. From $\mathcal{A}$'s key proof, $\mathcal{S}_1$ extracts dk_1 and subsequently recovers x_1 from $\mathcal{A}$'s ciphertexts, then registers $X = X_0 \cdot X_1$ with $\mathcal{F}_{\text{2ECDSA}}$.

In the Presigning phase, $\mathcal{S}_1$ samples $h \leftarrow \mathbb{Z}_q$, sets $H = G^h$, and simulates ϕ_0 for statement H. Upon receiving (com_k, com_γ, R) from $\mathcal{A}$, it computes $r' = R|_{\text{x-axis}}$ and sends $(\mathsf{set\text{-}nonce}, \mathsf{psid}, R)$ to $\mathcal{F}_{\text{2ECDSA}}$.

In the Signing phase, upon receiving $(C_\chi, com_\zeta, com_{\chi,v}, \delta, \pi_\delta, \pi_\chi, m)$ from $\mathcal{A}$, $\mathcal{S}_1$ computes $\mu := \Pi_{\text{CL}}.\mathsf{Dec}(dk_0; C_\chi^{\delta^{-1}})$, queries $\mathcal{F}_{\text{2ECDSA}}$ with $(\mathsf{sign}, \mathsf{psid}, m)$ to obtain (r', s^*), sets $\alpha := s^* \cdot \mu^{-1} \mod q$, and returns $s := \alpha \cdot \mu = s^*$ to $\mathcal{A}$.

Indistinguishability. (i) The simulated (ψ_0, ϕ_0) are indistinguishable from real proofsby zero-knowledge. (ii) The choice of W_0 is hidden by CL-IND-CPA and perfect hidng of commitments. (iii) The value α is determined post-hoc; $\mathcal{A}$ observes only $H = G^h$ and cannot verify $h = \alpha^{-1}$ without solving discrete logarithm. (iv) If $\mathcal{A}$ behaves honestly, soundness of the verified relations ensures $\mu = (m + r'x)/k$, so (r', s^*) is a valid ECDSA signature. Therefore, real and ideal executions are indistinguishable up to negligible probability.

□

Case C: Both Parties Honest.

Lemma 4. *If both parties are honest, then for all PPT distinguishers $\mathcal{Z}$, $\left|\Pr[\text{EXEC}^{\text{real}}_{\Pi,\mathcal{Z}} = 1] - \Pr[\text{EXEC}^{\text{ideal}}_{\mathcal{F}_{2ECDSA},\mathcal{Z}} = 1]\right| \leq \epsilon$.*

Proof. When neither party is corrupted, the adversary $\mathcal{A}$ has no internal access to any party's state and can only observe message transmitted over the authenticated channels. In UC terminology, this corresponds to a *passive eavasdropper* who sees the protocol transcript but cannot inject or modify messages.

We use a standard hybrid argument on the real transcript H_0.

$H_0 \Rightarrow H_1$: Replace every ZK proof by the simulator's output. By (computational) zero-knowledge the view is indistinguishable.

$H_1 \Rightarrow H_2$: Replace all Pedersen (vector) commitments by random group elements consistent with their public statements. Pedersen commitments are *perfectly hiding*, hence the two distributions are identical.

$H_2 \Rightarrow H_3$: Replace CL plaintexts by zeros (respecting homomorphic structure). By CL-IND-CPA the distributions are computationally indistinguishable.

In H_3 the transcript is independent of $x_0, x_1, k, \gamma, \chi, v$; the ideal execution reveals the same public outputs (X) and (r, s). Therefore H_3 and the ideal world are indistinguishable, which proves the claim. □

Proof of Theorem 1. Fix $\mathcal{A}$ and $\mathcal{Z}$. If P_0 is corrupted, apply Lemma 2; if P_1 is corrupted, apply Lemma 3; if none is corrupted, apply Lemma 4. In each case the distinguishing advantage is negligible, hence the UC-realization statement follows.

Security Properties (from the Ideal World) From the realization of $\mathcal{F}_{\text{2ECDSA}}$ we immediately obtain:

- **Hiding.** The transcript leaks nothing about $x_0, x_1, k, \gamma, \chi, v$ beyond the public key and the final signature, since $\mathcal{F}_{\text{2ECDSA}}$ only outputs (X) and issued signatures.
- **Unforgeability.** Any PPT adversary outputting a valid signature on a new message when at least one party is honest would contradict the *Verification* rule of Figure 2, which returns 0 on any pair not in the functionality's log. A successful real-world forger thus yields an EUF-CMA forger for single-party ECDSA.

4 ART-ECDSA Scheme

In this section, we extend the two-party construction from Section 3 to the general (n, t)-threshold setting, where one hardware party P_0 interacts with $n-1$ online cosigners $P_1, \ldots, P_{n-1}$. We denote the set of online parties by $\infty = [n] \setminus \{0\}$. The core asymmetric paradigm remains unchanged: P_0 performs minimal computation – generating the α-handle $H = G^{\alpha^{-1}}$, decrypting the final aggregated ciphertext, and outputting the signature – while all heavy MtA operations, commitment-based protocols, and multi-party coordination are handled by the online cosigners. In out design, P_1 acts as a gateway that coordinates protocols among online parties, aggregates intermediate results, and forwards only necessary messages to/from P_0; the remaining parties $P_2, \ldots, P_{n-1}$ participate in distributed protocols (DRG, MtA) but never communicate directly with P_0. Our scheme adopts TX25's robust three-round structure but fundamentally differs in computational distribution: while TX25 requires all parties to perform identical heavy operations in a fully connected network, we offload all such work to the online parties while maintaining a star topology centered at P_1.

DKG and setup. The DKG phase establishes the shared ECDSA public key X following the DRG protocol from Section 2.4, adapted to our asymmetric communication model. Each party generates a CL encryption key pair (ek_i, dk_i) for receiving encrypted shares in later protocols, and P_1 generates Pedersen commitment parameters $ck, \hat{ck}$ for binding commitments throughtout the scheme. Following [TX25], we invoke the DRG protocol based on Cascudo-David's PVSS [CD24] to generate t-threshold shares $\{x_i\}_{i \in S}$ of the signing key x, where each party's share x_i is the sum of encrypted contributions from all parties. Crucially, all online parties receive and store $W_0 = \Pi_{\text{CL}}.\mathsf{Enc}(ek_0; x_0)$, the CL encryption of P_0's share under P_0's public key, which will be used for homomorphic operations in the signing phase. At the end of this phase, each party stores its secret share x_i, the public key $X = \prod X_i^{\lambda_{i,S}}$, the CL public keys $\{ek_i\}_{i \in S}$, and W_0, with public verifiability ensuring that any malformed contributions are detected and excluded from S.

Presinging (message-independent). The presigning phase generates presignatures that can be cached for future signing requests, shifting all heavy computation away from the online signing phase. P_0 initiates by sampling $\alpha \leftarrow \mathbb{Z}_q$ and sending the α-handle $H = G^{\alpha^{-1}}$ with proof ϕ_0 to P_1, who broadcasts it to all online parties. This handle allows the online parties to compute $R = H^k$, deferring the final correction step (multiplication by α) to P_0 during signing; the effective nonce becomes $k' = k/\alpha$, and P_0's multiplication by α in the signing phase restores the correct signature equation.

A subtle but important difference from [TX25] arises from our asymmetric participation model. In DKG, the key shares $\{x_i\}_{i \in S}$ are generated over the full participant set S including P_0, using Lagrange coefficients $\lambda_{i,S}$. However, in presigning and signing, only the online parties $\infty = S \setminus \{0\}$ participate in the MtA protocols. Since Lagrange interpolation

coefficients depend on the interpolating set, directly using x_i in the MtA would yield incorrect recombination when P_0's share x_0 is later incorporated homomorphically via W_0. To address this, each online party P_i computes an adjusted share $x'_i = (\lambda_{i,S}/\lambda_{i,\infty}) \cdot x_i$ before entering the MtA for $x\gamma$. This rescaling ensures that when the online parties' contributions are combined with W_0^v (which encodes x_0), the Lagrange interpolation correctly reconstructs $x = \sum_{i \in S} \lambda_{i,S} \cdot x_i$.

The online parties then execute a two-round protocol combining DRG for t-threshold shares of k and multi-party MtA for shares of $k\gamma$ and $x'\gamma$, following the framework established in [TX25]. At the end of this phase, P_1 sends to P_0 only the commitments $\{\mathrm{com}_{k_i}, \mathrm{com}_{\gamma_i}\}_{i \in \infty}$ and the public nonce R, which P_0 stores for the signing phase. These commitments bind each party to their nonce k_i and temporary secret γ_i, enabling P_0 to later verify correctness of the masked shares without participating in the heavy presigning computatioin.

Singing (single online-to-hardware request). The signing phase completes signature generation with minimal involvement from P_0, requring only a single message exchange between P_1 and the hardware party. Given a message m, each online party P_i computes masked shares $\delta_{i,j}$ and $\chi_{i,j} = m\gamma_i + r\zeta_{i,j}$ for all $i, j \in \infty$, where $\zeta_{i,j}$ are the MtA output shares of $x'\gamma$ rom presigning. To enable public verificability, each party also computes $D_i = R^{\gamma_i}$ and $\Gamma_i = (G^m \cdot X^r)^{\gamma_i}$, then broadcasts these values along with a DDH-relation proof ϕ_i''' demonstrating consistency. The online parties exchange their masked shares and verify each other's proofs before proceediing to aggregation.

P_1 then aggregates all contributions using Lagrange interpolation. Specifically, P_1 computes $\delta = \sum_{i,j \in \infty} \lambda_{i,\infty}\lambda_{j,\infty}\delta_{i,j} = k\gamma$ and $\chi = \sum_{i,j \in \infty} \lambda_{i,\infty}\lambda_{j,\infty}\chi_{i,j}$, which represents $\gamma(m + rx)$ computed over the online parties' adjusted shares. To incorporate P_0's key share homomorphically, P_1 constructs $C_\chi = \Pi_{\mathrm{CL}}.\mathsf{Enc}(ek_0; \chi) \oplus W_0^v$ where $v = r\gamma \cdot \lambda_{0,S}$ is the properly scaled coefficient. This ciphertext encrypts $\gamma(m + r\sum_{i \in \infty} \lambda_{i,\infty} x'_i) + r\gamma\lambda_{0,\S}x_0$ $= \gamma(m + rx)$, the complete signautre numerator including P_0's contribution, P_1 also assembles aggregated proofs π_χ and π_δ from individual proofs $\{\pi_{\chi_i}, \pi_{\delta_i}$ by combining the commitment and response components under the shared challenge, enabling efficient batch verification.

P_1 forwards $(C_\chi, \mathrm{com}_\zeta, \mathrm{com}_\chi, v, \delta, \pi_\chi, \pi_\delta, m, r)$ to P_0.The hardware party reconstructs the aggregated commitments $\mathrm{com}_k = \prod_{j \in \infty} \mathrm{com}_{k_j}^{\lambda_{j,\infty}}$ and $\mathrm{com}_\gamma = \prod_{j \in \infty} \mathrm{com}_{\gamma_j}^{\lambda_{j,\infty}}$ from the values stored during presigning, then verifies π_χ and π_δ against theses commitments. Upon successful verification, P_0 decrypts $s' = \Pi_{\mathrm{CL}}.\mathsf{Dec}(dk_0; C_\chi)$ and computes $s = s' \cdot \alpha \cdot \delta^{-1}$. By construction, this yields $s = \alpha \cdot \gamma(m + rx)/k\gamma) = (m + rx)/(k/\alpha) = (m + rx)/k'$, a valid ECDSA signature with effective nonce $k' = k/\alpha$. P_0 performs a final self-check by verifying $\Pi_{\mathrm{ECDSA}}.\mathrm{Verify}(m, r, s)$ before returning s to P_1.

If verification fails at any point, P_1 coordinates cheater identification without involving P_0. Since each online part's contribution is accompanied by individual proofs π_{χ_i} and π_{δ_i}, P_1 can verify each party's proof separately to identify the malicious participant. The identified cheater is excluded from S, and as long as $|S| \geq t$ after exclusing, the signature can be reassembled through Lagrange recombination over the remaining honest shares. This achieves robustness inherited from [TX25] while keeping P_0 entirely out of the dispute resolution process– a key advantage of our asymmetric design.

4.1 Security Proof

We now extend the two-party security result to the (n, t)-threshold setting, where one hardware party P_0 interacts with $n-1$ online cosigners $P_1, \ldots, P+n-1$. Let $\infty := [n]\backslash\{0\}$ denote the set of online parties, and let $S \subseteq [n]$ with $|S| \geq t$ denote the subset participating in a signing session.

Proof Overview. At a high level, the protocol enforces correct signing behavior by constraining all actions of the online parties through zero-knowledge proofs, while party

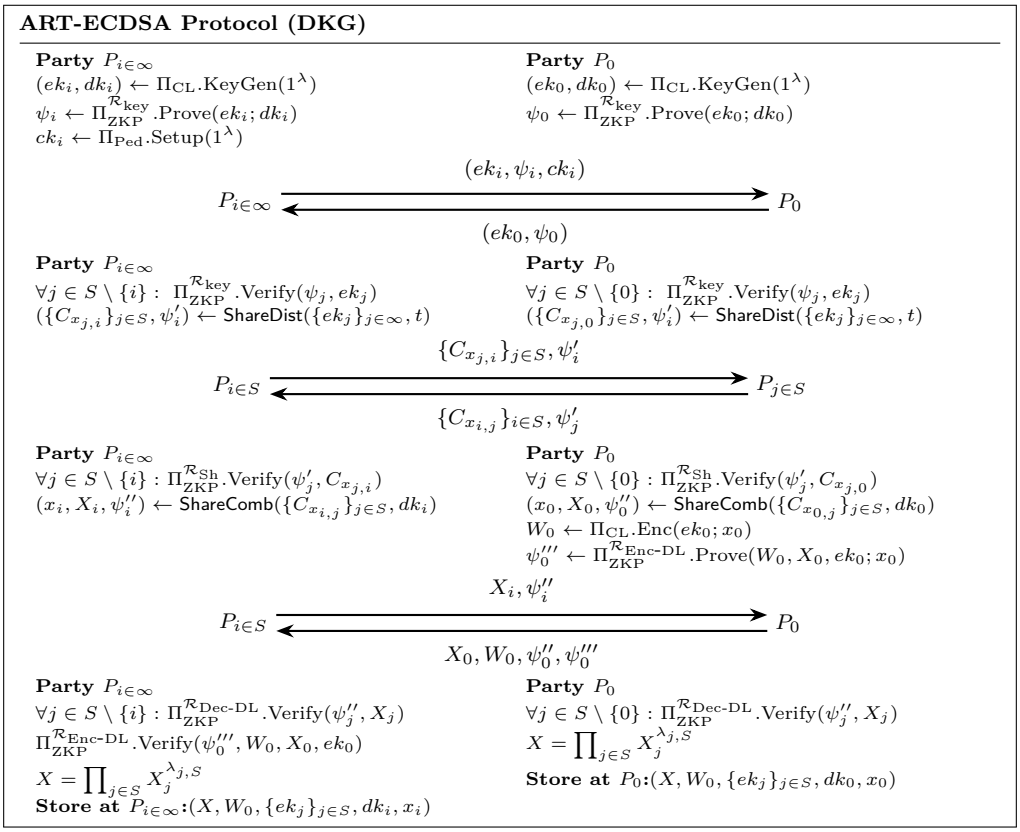

Figure 3: ART-ECDSA protocol (DKG).

P_0 only verifies these proofs and never reveals its secret state. As a result, any deviation by malicious online parties is either detected by proof soundness or yields no additional information due to zero-knowledge. The proof proceeds by viewing the online parties P_∞ as a single virtual party whose interface to P_0 is identical to P_1 in the two-party protocol, then applying the two-party security result via UC composition.

Ideal Functionality $\mathcal{F}_{\mathrm{tECDSA}}^{n,t}$ (From two-party to threshold functionality): For brevity, we formally present only the two-party ideal functionality in Figure 2, consisting of a hardware party P_0 and a single online party P_1. The threshold functionality $\mathcal{F}_{\mathrm{tECDSA}}^{n,t}$ is obtained by interpreting P_1 as a *virtual* party P_∞ that represents a set of online cosigners, while P_0 remains a distinguished party with a hardware-specific role.

Crucially, the threshold is defined over the *entire* party set $\mathcal{P} = P_0 \cup \mathcal{P}_\infty$: security and correctness hold as long as at least t parties in $\mathcal{P}$ are honest, i.e., fewer than t parties are corrupted. The asymmetry in our model affects only the protocol interface and division of computation, but not the underlying threshold assumption.

Operationally, P_∞ is merely an interface abstraction that collects signing requests and aggregates the protocol messages from the online cosigners. All security properties of the two-party functionality–unforgeabiltiy, robustness, and abort behavior–lift directly to the threshold setting under this interpretation.

4.1.1 Hybrid Model

Our proof proceeds in the $(\mathcal{F}_{\mathrm{MPC}}, \mathcal{F}_{\mathrm{zk}}^{R_{\mathrm{key}}})$-hybrid model, where:

- $\mathcal{F}_{\mathrm{MPC}}$ is an ideal MPC functionality that allows the online parties $P_\infty := \{P_i\}_{i\in\infty}$ to

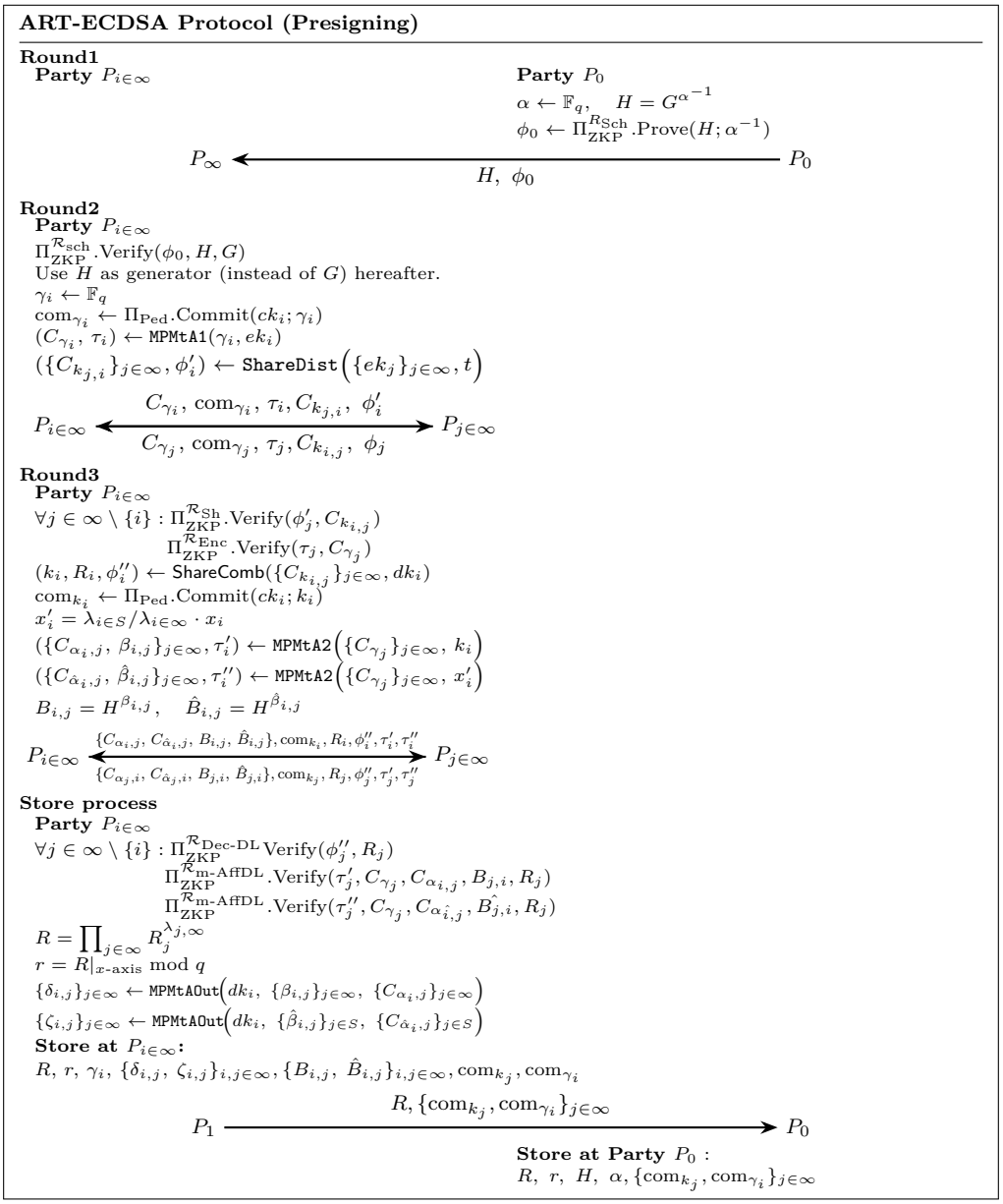

Figure 4: ART-ECDSA protocol (Presign).

jointly compute any function on their private inputs with guaranteed output delivery (assuming $n \geq wt - 1$ and at most $t - 1$ corruptions among online parties).

- $\mathcal{F}_{\text{zk}}^{R_{\text{key}}}$ provides simulation-extractable zero-knowledge proofs for the CL key generation relation.

4.1.2 Virtual Party Abstraction

The key insight is that the online parties P_∞ can be viewed as a single *virtual party* whose external interface to P_0 is identical to that of P_1 in the two-party protocol. Specifically, inside the MPC, the online parties jointly compute:

- **Lagrange recombinations:** $R = \prod_{i\in\infty} R_i^{\lambda_i}, \quad \gamma = \sum_{i\in\infty} \lambda_i \gamma_i, \quad k = \sum_{i\in\infty} \lambda_i k_i$

ART-ECDSA Protocol (Signing)

Party $P_{i\in\infty}$
Pick three random $(t-1)$-degree polynomials
$f_i(\cdot), f_i'(\cdot), f_i''(\cdot)$ with $f_i(0) = f_i'(0) = f_i''(0) = 0$
$\forall j:\ \delta_{i,j} \leftarrow \delta_{i,j} + f_i(j) \pmod q$
$\chi_{i,j} \leftarrow m \cdot \gamma_i + r \cdot \zeta_{i,j} + f_i'(j) \pmod q$
$v_{i,j} = r \cdot \gamma_i \cdot \lambda_{0,S} + f_i''(j)$
$D_i = R^{\gamma_i},\ \Gamma_i = (G^m \cdot X^r)^{\gamma_i}$
$\phi_i''' \leftarrow \Pi_{\text{ZKP}}^{\mathcal{R}_{\text{DDH}}}.\text{Prove}(R, D_i, G^m \cdot X^r, \Gamma_i; \gamma_i)$
$\chi_i = \sum_{j\in\infty} \lambda_{j,\infty}\chi_{i,j}$
$v_i = \sum_{j\in\infty} \lambda_{j,\infty} v_{i,j}$
$\zeta_i = \prod_{j\in\infty} \lambda_{j,\infty}\zeta_{i,j}$
$C_{\chi_i} = \Pi_{\text{CL}}.\text{Enc}(ek_0; \chi_i)$
$\text{com}_{\zeta_i} \leftarrow \Pi_{\text{Ped}}.\text{Commit}(ck; \zeta_i)$
$\text{com}_{\chi_i, v_i} \leftarrow \Pi_{\text{Ped-vec}}.\text{Commit}(\hat{ck}; \chi_i, v_i)$

$$P_{i\in\infty} \xrightarrow[\text{com}_{\chi_i,v_i}, \text{com}_{\zeta_i}, C_{\chi_i}]{D_i, \Gamma_i, \phi_i''', \{\delta_{i,j}, \chi_{i,j}, v_{i,j}\}_{j\in\infty}} P_{j\in\infty}$$

Party $P_i \in \infty$
$\text{com}_k = \prod_{j\in\infty} \text{com}_{k_i}^{\lambda_{j,\infty}}$
$\text{com}_\gamma = \prod_{j\in\infty} \text{com}_{\gamma_i}^{\lambda_{j,\infty}}$
$\text{com}_{\chi,v} \leftarrow \prod_{j\in\infty} \text{com}_{\chi_j,v_j}^{\lambda_{j,\infty}}$
$\text{com}_\zeta \leftarrow \prod_{j\in\infty} \text{com}_{\zeta_j}^{\lambda_{j,\infty}}$
$\text{com}_\chi = \text{com}_\zeta^r \cdot \text{com}_\gamma^m$
$\text{com}_v = \text{com}_\gamma^r$
$\delta \leftarrow \sum_{i,j\in\infty} \lambda_{i,\infty}\lambda_{j,\infty} \cdot \delta_{i,j} \pmod q$
$C_\chi \leftarrow \prod_{j\in\infty} C_{\chi_j}^{\lambda_{j,\infty}}$
$e_\chi \leftarrow H(C_\chi, \text{com}_{\chi,v}, \text{com}_\zeta, \text{com}_\gamma)$
$\pi_{\chi_i} \leftarrow \Pi_{\text{ZKP}}^{\mathcal{R}_\chi}.\text{Prove}(C_{\chi_i},\ \text{com}_{\chi_i,v_i},\ \text{com}_{\zeta_i},\ \text{com}_{\gamma_i}, e_\chi; \chi_i, v_i.r, \gamma_i)$
$e_\delta \leftarrow H(\text{com}_k, \text{com}_\gamma, \delta, H, R)$
$\pi_{\delta_i} \leftarrow \Pi_{\text{ZKP}}^{\mathcal{R}_\delta}.\text{Prove}(\text{com}_{k_i},\ \text{com}_{\gamma_i},\ \delta_i,\ H,\ R,\ e_\delta; k_i, \gamma_i)$

$$P_{i\in\infty\setminus\{1\}} \xrightarrow{\pi_{\chi_i}, \pi_{\delta_i}} P_1$$

Party P_1
$\forall j: D_j = \prod_{i\in\infty} (G^{\delta_{j,i}} \cdot B_{j,i}^{-1} \cdot B_{i,j})^{\lambda_{i,\infty}}$
$\Gamma_j = \prod_{i\in\infty} (G^{\chi_{j,i}} \cdot \hat{B}_{j,i}^{-r} \cdot \hat{B}_{i,j}^{r})^{\lambda_{i,\infty}}$
verify ϕ_j'''
$\chi = \sum_{i\in\infty}\sum_{j\in\infty} \lambda_{i,\infty} \cdot \lambda_{j,\infty} \cdot \chi_{i,j}$
$\delta = \sum_{i\in\infty}\sum_{j\in\infty} \lambda_{i,\infty} \cdot \lambda_{j,\infty} \cdot \delta_{i,j}$
$v = \sum_{i\in\infty}\sum_{j\in\infty} \lambda_{i,\infty} \cdot \lambda_{j,\infty} \cdot v_{i,j}$
$C_\chi = \Pi_{\text{CL}}.\text{Enc}(ek_0; \chi) \oplus W_0^v$
$\pi_\chi \leftarrow \Pi_{\text{ZKP}}^{R_\chi}.\text{Assemble}(C_\chi, \text{com}_{\chi,v}, \text{com}_\chi, \text{com}_v, \{\pi_{\chi_j}\}_{j\in\infty})$
$\pi_\delta \leftarrow \Pi_{\text{ZKP}}^{R_\delta}.\text{Assemble}(\text{com}_k, \text{com}_\gamma, \delta, H, R, \{\pi_{\delta_j}\}_{j\in\infty})$

$$P_1 \xrightarrow{C_\chi,\ \text{com}_\zeta,\ \text{com}_{\chi,v},\ \text{com}_\chi,\ \delta,\ \pi_\chi,\ m,\ r} P_0$$

Party P_0
$\text{com}_\chi = \text{com}_\zeta^r \cdot \text{com}_\gamma^m$
$\text{com}_v = \text{com}_\gamma^r$
$\text{com}_k = \sum_{j\in\infty} \lambda_{j,\infty}\text{com}_{k_i}$
$\text{com}_\gamma = \sum_{j\in\infty} \lambda_{j,\infty}\text{com}_{\gamma_i}$
$\Pi_{\text{ZKP}}^{R_\chi}.\text{Verify}(\pi_\chi, C_\chi, \text{com}_{\chi,v}, \text{com}_\chi, \text{com}_v)$
$\Pi_{\text{ZKP}}^{R_\delta}.\text{Verify}(\pi_\delta, \text{com}_k, \text{com}_\gamma, \delta, H, R)$
$s' = \Pi_{\text{CL}}.\text{Dec}(dk_0;\ C_\chi)$
$s = s' \cdot \alpha \cdot \delta^{-1}$
Assert!$(\Pi_{ECDSA}.\text{Verify}(m, r, s) == 1)$

$$P_1 \xleftarrow{s} P_0$$

Final signature: (r, s)

Figure 5: ART-ECDSA protocol (Sign).

- **MtA aggregations:** $\delta = \sum_{i,j\in\infty} \lambda_i\lambda_j\delta_{i,j} = \gamma \cdot k, \quad \chi = \sum_{i,j\in\infty} \lambda_i\lambda_j\chi_{i,j} = \gamma(m + rx)$
- **Ciphertext construction:** $C_\chi = W_0^v \oplus \Pi_{\mathrm{CL}}.\mathsf{Enc}(ek_0, \chi)$ where $v = r\gamma$
- **Commitment aggregations:** $com_k = \prod_{j\in\infty} com_{k_k}^{\lambda_j}, \quad com_\gamma = \prod_{j\in\infty} com_{\gamma_j}^{\lambda_j}$
- **Proof assembly:** Aggregate individual proofs $\{\pi_{\chi_i}, \pi_{\delta_i}\}_{i\in\infty}$ into combined proofs π_χ, π_δ.

The virtual party P_∞ releases to P_0 only the public artifacts expected in the two-party protocol: $(C_\chi, com_\zeta, com_{\chi,v}, \delta, \pi_\chi, \pi_\delta, m, r)$ and outputs $\mathsf{Identify}(i)$ upon any failed internal verification.

4.1.3 Main Theorem

Theorem 2 (Multi-Party Security)**.** *Assume DDH, CL-IND-CPA, HSM (Hard Subgroup Membership), EUF-CMA of ECDSA, and simulation-extractable ZKP in the ROM. Let the online parties execute any UC-secure MPC tolerating up to $t-1$ corruptions with guaranteed output deliver. Then the multi-party protocol $\Pi^{n,t}$ UC-realizes $\mathcal{F}_{tECDSA}^{n,t}$ with robustness and identifiable abort, provided $n \geq 2t-1$.*

Proof. We construct a simulator $\mathcal{S}$ for each corruption scenario and show indistinguishability from the ideal world via a sequence of hybrid arguments.

Corruption Scenarios. Let $S_c \subseteq [n]$ denote the set of corrupted parties with $|S_c| \leq t-1$. We distinguish three cases based on whether $P_0 \in S_c$:

- **Case A:** $P_0 \in S_c$ (hardware party corrupted, some online parties honest)
- **Case B:** $P_0 \notin S_c$ (hardware party honest, some online parties corrupted)
- **Case C:** $S_c = \emptyset$ (all parties honest)

Case A: Malicious P_0. *Simulator $\mathcal{S}_A$: $\mathcal{S}_A$* simulates the honest online parties the honest online parties $\{P_i\}_{i\in\infty\setminus S_c}$ to $\mathcal{A}$.
Key Generation: For each honest P_i, generate CL key-pair (ek_i, dk_i) and simulate proofs. Using SE-ZKP, extract dk_0 from $\mathcal{A}$'s key proof and subsequently recover x_0 from $\mathcal{A}$'s encrypted shares. For corrupted online parties $P_j \in S_c \cap \infty$, extract x_j similarly. With $\{x_j\}_{j\in S_c}$ known, $\mathcal{S}_A$ can interpolate to determine the honest parties' public shares $\{X_i\}_{i\notin S_c}$ consistent with the public key X received from $\mathcal{F}_{\mathrm{tECDSA}}^{n,t}$.
Presigning: Receive H, ϕ_0 from $\mathcal{A}$ (as P_0). Extract α via SE-ZKP. For honest online parties, execute the DRG and MtA protocols honestly, computing $\{k_i, \gamma_i, \delta_{i,j}, \zeta_{i,j}\}$ as prescribed. Compute $R = H^k$ where $k = \sum_{i\in\infty} \lambda_i k_i$ and send $(\mathsf{set\text{-}nonce}, \mathsf{psid}, R)$ to $\mathcal{F}_{\mathrm{tECDSA}}^{n,t}$.
Singing: Given message m, compute all signature components honestly using the extracted α. By Lemma 1, the resulting (r, s) is a valid ECDSA signature, matching $\mathcal{F}_{\mathrm{tECDSA}}^{n,t}$'s output.
Indistinguishability: By SE-ZKP, the simulated proofs are indistinguishable from real proofs. All values computed by honest parties follow the protocol specification exactly. The MPC among online parties is UC-secure so $\mathcal{A}$'s view is indistinguishable.

Case B: Malicious Online Parties (Honest P_0).
Simulator $\mathcal{S}_B$: $\mathcal{S}_B$ simulates the honest P_0 and honest online parties $\{P_i\}_{i\in\infty\setminus S_c}$.
Key Generation: Generate fresh CL key-pair (ek_0, dk_0) for P_0 and set W_0 arbitrarily. Simulate ψ_0 using the ZK simulator. For honest online parties, generate keys honestly. Extract $\{x_j, dk_j\}_{j\in S_c\cap\infty}$ from corrupted parties' proofs and ciphertexts. Register X with $\mathcal{F}_{\mathrm{tECDSA}}^{n,t}$.
Presigning: For P_0: sample $h \leftarrow \mathbb{Z}_q$, set $H = g^h$, simulate ϕ_0. For honest online parties: execute DRG and MtA using random k_i^*, x_i^* instead of correct shares (following TX25's simulation strategy). Upon receiving R determined by the protocol execution, send $(\mathsf{set\text{-}nonce}, \mathsf{psid}, R)$ to $\mathcal{F}_{\mathrm{tECDSA}}^{n,t}$.

Singing: Query $\mathcal{F}_{\text{tECDSA}}^{n,t}$ with $(\mathsf{sign}, \mathsf{psid}, m)$ to obtain (r, s^*). Compute $\mu := \Pi_{\text{CL}}.\mathsf{Dec}(dk_0; C_\chi^{\delta^{-1}})$ from the aggregated ciphertext. Set $\alpha := s^* \cdot \mu^{-1} \mod q$ and return $s := \alpha \cdot \mu = s^*$.

For honest online parties, simulate their shares $\{\delta_{i,j}, \chi_{i,j}\}$ using dummy values (a, b) where a and b are t-threshold shares of dummy signing key and nonce satisfying $s^* = (m + r \cdot a)/b$. This follows the TX25 simulation technique.

Indistinguishability: (i) By CL-IND-CPA, $W_0 = \mathsf{Enc}(ek_0; 0)$ is indistinguishable from $\mathsf{Enc}(ek_0; x_0)$. (ii) By ZK property, simulated proofs are indistinguishable. (iii) By security of CL affine operations (Theorem 1 of TX25), using k_i^*, x_i^* instead of correct shares is undetectable. (iv) α is determined post-hoc; $\mathcal{A}$ sees only $H = G^h$ and cannot verify $h = \alpha^{-1}$ without solving Discrete Logarithm. (v) By UC-security of the underlying MPC, the corrupted parties' view in the real and simulated executions is indistinguishable.

Case C: All Parties Honest.

When no party is corrupted we use a standard hybrid argument.

H_0: Real execution.

$\mathsf{H}_0 \Rightarrow \mathsf{H}_1$: Replace all ZK proofs with simulator outputs. Indistinguishable by computational zero-knowledge.

$\mathsf{H}_1 \Rightarrow \mathsf{H}_2$: Replace Pedersen commitments with random group elements. Identical distributions by perfect hiding.

$\mathsf{H}_2 \Rightarrow \mathsf{H}_3$: Replace CL ciphertext plaintexts with zeros. Indistinguishable by CL-IND-CPA.

In H_3, the transcript is independent of $\{x_i, k_i, \gamma_i, \delta_{i,j}, \zeta_{i,j}\}_{i,j \in \infty}$. The ideal execution reveals only (X) and (r, s), matching H_3.

Robustness and Identifiable Abort. Every protocol message is accompanied by a publicly verifiable proof. If verification fails for party P_j, all honest parties output $\mathsf{Identify}(j)$ and exclude P_j from S. Since $n \geq 2t - 1$, even after removing up to $t - 1$ cheaters, at least t honest parties remain, ensuring signature completion. This matches $\mathcal{F}_{\text{tECDSSA}}^{n,t}$'s robustness guarantee.

Composition. By UC composition, replacing the real online-party execution with $\mathcal{F}_{\text{MPC}}$ changes the environment's view by at most $\mathsf{Adv}_{\text{MPC}}(\lambda)$. After this virtualization, the execution reduces to the two-party setting between P_0 and P_∞. Applying Theorem 1 yields:

$$\left|\Pr[\mathsf{EXEC}_{n,t}^{\text{real}} = 1] - \Pr[\mathsf{EXEC}_{n,t}^{\text{ideal}} = 1]\right| \leq \mathsf{Adv}_{\text{MPC}}(\lambda)\mathsf{Adv}_{\text{2P}}(\lambda) = \epsilon(\lambda).$$

Therefore, $\Pi^{n,t}$ UC-realizes $\mathcal{F}_{\text{tECDSA}}^{n,t}$ with robustness and identifiable abort. □

5 Implementation

We implemented all protocols in Rust, using the GMP library for arbitrary-precision arithmetic. The Castagnos-Laguillanumie (CL) encryption is instantiated the CL-HSM_q using a 256-bit plaintext space $\mathbb{Z}_q$ and discriminant $\Delta_k = 1827$ bits, following the parameters of [TX25]. For comparison, the Paillier-based scheme of [BMP22] uses an RSA modulus of 3072 bits to achieve equivalent security. Elliptic curve operations use the secp256k1 curve, targeting 128-bit security.

Experimental Setup. We evaluate online cosigners on a MacBook M1 Pro with 16 GB RAM. The hardware party P_0 runs on an ARM Cortex-M7 microcontroller clocked at 400MHz with 3 MB Flash and 2 MB SRAM, representative of secure elements used in hardware wallets. The [BMP22] baseline was evaluated using the authors' reference implementation [Ped91]. All schemes share identical elliptic-curve and hashingg primitives for fair comparison.

Table 3: Computation time and communication cost comparison for threshold ECDSA schemes.

Threshold	Scheme	Computation Time (s)				Communication Size (KB)			
		Online Party		HW Party		Online ↔ Online		Online ↔ HW	
		Pre-sign	Sign	Pre-sign	Sign	Pre-sign	Sign	Pre-sign	Sign
$t = 2$	[BMP22]	0.06	0.12	243	244	–	–	5.19	2.73
	[TX25]	0.97	<0.01	98.27	0.48	–	–	30.74	0.93
	ART-ECDSA (2-party)	<0.01	0.13	0.05	7.65	–	–	0.14	2.86
$t = 3$	[BMP22]	0.11	0.53	246	244	7.72	32.57	10.39	2.73
	[TX25]	1.62	<0.01	168.89	0.82	78.46	2.26	78.46	2.26
	ART-ECDSA (threshold)	0.71	0.09	0.05	9.05	18.72	10.17	0.17	2.91
$t = 4$	[BMP22]	0.15	0.89	247	244	10.29	51.96	13.46	2.73
	[TX25]	1.94	<0.01	204.59	1.07	110.50	3.11	110.50	3.11
	ART-ECDSA (threshold)	1.05	0.10	0.05	9.07	36.19	11.13	0.23	2.94
$t = 5$	[BMP22]	0.19	1.34	248	244	12.86	73.56	16.53	2.73
	[TX25]	2.26	<0.01	240.75	1.42	148.00	4.11	148.00	4.11
	ART-ECDSA (threshold)	1.37	0.12	0.05	9.18	62.65	12.28	0.30	2.98

Results. Table 3 summarizes computation time and communication cost for the three major phases across varying thresholds t. For our asymmetric protocol, results are reported separately for the hardware party (P_0) and the online cosigners (P_∞).

Compuation. The hardware party achieves dramatic efficiency gains in presigning. In the two-party setting ($t = 2$), P_0 completes presigning in under 10 ms –over 24,000x faster than [BMP22] (243 s) and 9,800x faster than [TX25] (98.27 s). At $t = 5$, P_0 requires only 50 ms, achieving 4,960x speedup over [BMP22] (248 s) and 4,815x speedup over [TX25] (240.75 s). In the signing phase, P_0 completes in 7.65-9.18 s on the constrained Cortex-M7, representing a 26-32x improvement over [BMP22]'s 244 s. The online cosigners in our scheme also outperform [TX25] by approximately 30%, as they execute MtA with one fewer party.

Communication. P_0's communication overhead is minimal. At $t = 2$, presigning requires only 0.14 KB – 37x less than [BMP22] (5.19 KB) and 220x less than [TX25] (30.74 KB). At $t = 5$, the gap widens dramatically: P_0 transmits only 0.30 KB, compared to 16.53 KB for [BMP22] (55x reduction) and 148 KB for [TX25] (over 493x reduction). Signing communication remains under 3 KB across all thresholds, easily fitting within BLE and NFC bandwidth constraints.

6 Conclusion

We presented **ART-ECDSA**, the first robust and hardware-friendly threshold ECDSA scheme designed explicitly for asymmetric settings where one participant is a resource-constrained hardware device. Building on the robustness principles of modern symmetric threshold designs and the efficiency insights of asymmetric models, ART-ECDSA achieves constant-time computation for the hardware signer while maintaining provable security and full robustness under the UC framework.

Our implementation and benchmarks demonstrate that ART-ECDSA achieves substantial performance gains over prior works. In particular, the hardware party P_0 , running on an ARM Cortex-M7 microcontroller (400 MHz, 3 MB Flash, 2 MB SRAM), performs only lightweight operations (approximately 50 ms in the presigning phase and ≤ 10 s in the signing phase), while the overall communication remains compact – about 300 Bytes during presigning and 3 KB during signing – which comfortably fits within typical BLE or NFC bandwidth.

In conclusion, our prototype confirms that ART-ECDSA achieves the targeted design goals: (i) constant-time lightweight workload for the hardware device, (ii) robustness

and cheater identification inherited from [TX25], and (iii) communication well within low-bandwidth channels such as BLE or NFC.

References

[BMP22] Constantin Blokh, Nikolaos Makriyannis, and Udi Peled. Efficient asymmetric threshold ecdsa for mpc-based cold storage. *Cryptology ePrint Archive*, 2022.

[CD24] Ignacio Cascudo and Bernardo David. Publicly verifiable secret sharing over class groups and applications to dkg and yoso. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 216–248. Springer, 2024.

[CGG+20] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. Uc non-interactive, proactive, threshold ecdsa with identifiable aborts. *CCS '20: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 1769–1787, 2020.

[CL15] Guilhem Castagnos and Fabien Laguillaumie. Linearly homomorphic encryption from ddh. *Topics in Cryptology - CT-RSA 2015*, pages 487–505, 2015.

[DKLS18] Jack Doerner, Yashvanth Kondi, Eysa Lee, and Abhi Shelat. Secure two-party threshold ecdsa from ecdsa assumptions. *2018 IEEE Symposium on Security and Privacy (SP)*, pages 980–997, 2018.

[DKLS19] Jack Doerner, Yashvanth Kondi, Eysa Lee, and Abhi Shelat. Threshold ecdsa from ecdsa assumptions: The multiparty case. *2019 IEEE Symposium on Security and Privacy (SP)*, pages 1051–1066, 2019.

[DKLS24] Jack Doerner, Yashvanth Kondi, Eysa Lee, and Abhi Shelat. Threshold ecdsa in three rounds. *2024 IEEE Symposium on Security and Privacy (SP)*, pages 3053–3071, 2024.

[GG18] Rosario Gennaro and Steven Goldfeder. Fast multiparty threshold ecdsa with fast trustless setup. *CCS '18: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 1179–1194, 2018.

[GG20] Rosario Gennaro and Steven Goldfeder. One round threshold ecdsa with identifiable abort. *Cryptology ePrint Archive*, 2020.

[JMV01] Don Johnson, Alfred Menezes, and Scott Vanstone. The elliptic curve digital signature algorithm (ecdsa). *International Journal of Information Security*, 1:36–63, 2001.

[MYG24] Nikolaos Makriyannis, Oren Yomtov, and Arik Galansky. Practical key-extraction attacks in leading mpc wallets. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 3053–3064, 2024.

[Ped91] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Annual international cryptology conference*, pages 129–140. Springer, 1991.

[TX25] Guofeng Tang and Haiyang Xue. Robust threshold ecdsa with online-friendly design in three rounds. *2025 IEEE Symposium on Security and Privacy (SP)*, pages 203–221, 2025.